Ejercicio 1: Ordenamiento Burbuja paso a paso

Prompt redactado:

Algoritmo a utilizar: Ordenamiento por Burbuja (Bubble Sort).

Razonamiento del algoritmo: Explica cómo funciona el algoritmo de burbuja, detallando por qué el elemento de mayor valor de cada sublista no ordenada va "flotando" o desplazándose paulatinamente hacia la última posición disponible al comparar e intercambiar elementos adyacentes continuos.

Datos que recibe: Un arreglo desordenado de números enteros (ejemplo: [5, 3, 8, 1, 2]).

Lo que debe mostrar por pantalla:

1. El arreglo original antes de comenzar.
2. El estado completo del arreglo inmediatamente después de finalizar cada pasada del ciclo externo.
3. El arreglo final totalmente ordenado.

Comentarios que debe incluir el código: Comentarios explicando el propósito de los dos bucles anidados y la condición de intercambio de elementos adyacentes.

Verificación del resultado: Imprimir el arreglo paso a paso para confirmar que al término de la pasada k, los k elementos más grandes se ubican de manera definitiva en el extremo derecho.

Ejercicio 2: Comparación entre Burbuja y Selección

Prompt redactado:

Algoritmo a utilizar: Implementación comparativa de bubbleSort(int[] array) y selectionSort(int[] array).

Razonamiento del algoritmo: Explica la diferencia conceptual entre la estrategia de Bubble Sort (comparación continua de vecinos inmediatos con intercambios frecuentes) y Selection Sort (búsqueda directa del elemento mínimo en el resto del arreglo para realizar un único intercambio por pasada).

Datos que recibe: Dos copias exactas del mismo arreglo desordenado de enteros para asegurar una prueba justa.

Lo que debe mostrar por pantalla:

1. Para Bubble Sort: cantidad total de comparaciones y cantidad total de intercambios realizados.
2. Para Selection Sort: cantidad total de comparaciones y cantidad total de intercambios realizados.

3. Una breve conclusión comparando la eficiencia de ambos métodos basándose en las métricas obtenidas.

Comentarios que debe incluir el código: Comentarios identificando las variables contadoras dentro de las estructuras iterativas de ambos métodos.

Verificación del resultado: Validar que ambos métodos produzcan exactamente el mismo arreglo ordenado partiendo del mismo estado inicial.

Ejercicio 3: Ordenamiento por Inserción con arreglo casi ordenado

Prompt redactado:

Algoritmo a utilizar: Ordenamiento por Inserción (Insertion Sort).

Razonamiento del algoritmo: Explica por qué Insertion Sort resulta excepcionalmente eficiente (alcanzando una complejidad cercana a $O(n)$) cuando se ejecuta sobre un arreglo previamente o casi ordenado, fundamentando esto en la interrupción temprana del bucle interno de desplazamiento.

Datos que recibe: Un arreglo casi ordenado de números enteros (ejemplo: int[] numeros = {1, 2, 3, 5, 4, 6, 7};).

Lo que debe mostrar por pantalla:

1. El arreglo original.
2. La cantidad total de desplazamientos/movimientos de elementos realizados durante el proceso de inserción.
3. El arreglo final ordenado.

Comentarios que debe incluir el código: Comentarios que destaquen la condición de parada del bucle while o for interno al encontrar un elemento menor o igual.

Verificación del resultado: Constatar que el número total de desplazamientos sea mínimo en comparación con el tamaño del arreglo.

Ejercicio 4: Ordenamiento de nombres con Selection Sort

Prompt redactado:

Algoritmo a utilizar: Ordenamiento por Selección (Selection Sort) para cadenas de texto (String).

Razonamiento del algoritmo: Razona cómo cambia la lógica de comparación cuando se ordenan cadenas alfabéticamente en lugar de valores numéricos simples, justificando el uso obligatorio del método .compareTo() o

.compareToIgnoreCase() de la clase String de Java para determinar el orden lexicográfico.

Datos que recibe: Un arreglo de nombres tipo String desordenado (ejemplo: String[] nombres = {"Lucia", "Ana", "Pedro", "Juan"};).

Lo que debe mostrar por pantalla:

1. El arreglo de nombres original.
2. El arreglo de nombres ordenado alfabéticamente de la 'A' a la 'Z'.

Comentarios que debe incluir el código: Comentarios explicando cómo el método compareTo devuelve valores negativos, cero o positivos para indicar precedencia alfabética.

Verificación del resultado: Comprobar visualmente que el arreglo final mantenga un orden alfabético estricto.

Ejercicio 5: ShellSort explicando los gaps

Prompt redactado:

Algoritmo a utilizar: Ordenamiento Shell (ShellSort).

Razonamiento del algoritmo: Explica por qué ShellSort mejora ostensiblemente el rendimiento de Insertion Sort al permitir comparaciones e intercambios de elementos distantes mediante un salto o gap descendente, reduciendo desordenes masivos antes de aplicar la pasada final de $\text{gap} = 1$.

Datos que recibe: Un arreglo de enteros desordenado.

Lo que debe mostrar por pantalla:

1. El valor del gap utilizado en cada etapa de la secuencia.
2. El estado del arreglo inmediatamente después de completar las ordenaciones asociadas a cada gap.
3. El arreglo final completamente ordenado.

Comentarios que debe incluir el código: Comentarios indicando la secuencia de reducción del gap (por ejemplo, gap = gap / 2) y su impacto en las sublistas.

Verificación del resultado: Confirmar que al llegar a $\text{gap} = 1$, el arreglo ya se encuentre casi totalmente ordenado, haciendo esa última fase sumamente rápida.

Ejercicio 6: Quicksort con primer elemento como pivote

Prompt redactado:

Algoritmo a utilizar: Quicksort basado en la estrategia Divide y Vencerás con llamada recursiva.

Razonamiento del algoritmo: Relaciona el concepto de recursividad y el enfoque Divide y Vencerás con el funcionamiento de Quicksort, explicando cómo el elemento pivote particiona el problema en dos subproblemas independientes de menor tamaño.

Datos que recibe: Un arreglo de enteros desordenados, fijando siempre como pivote el primer elemento de la subpartición activa.

Lo que debe mostrar por pantalla en cada etapa:

1. El valor del pivote elegido.
2. El subarreglo de elementos menores a la izquierda.
3. El subarreglo de elementos mayores/iguales a la derecha.
4. Al finalizar todas las llamadas recursivas, el arreglo ordenado definitivo.

Comentarios que debe incluir el código: Comentarios explicando la condición de corte de la función recursiva y el proceso de particionado.

Verificación del resultado: Verificar que cada elemento pivote quede posicionado en su lugar relativo correcto dentro del arreglo.

Ejercicio 7: Peor caso de Quicksort

Prompt redactado:

Algoritmo a utilizar: Quicksort (utilizando el primer elemento como pivote) aplicado sobre el peor escenario.

Razonamiento del algoritmo: Explica conceptualmente por qué elegir invariablemente el primer elemento como pivote cuando los datos ya se encuentran totalmente ordenados genera la peor complejidad temporal esperada ($O(n^2)$), debido a que produce particiones completamente desbalanceadas (una vacía y otra de tamaño $n - 1$).

Datos que recibe: Un arreglo que ya está ordenado de menor a mayor (ejemplo: int[] numeros = {1, 2, 3, 4, 5, 6, 7};).

Lo que debe mostrar por pantalla:

1. La cantidad total de llamadas recursivas realizadas durante el proceso.
2. El arreglo final resultante.

Comentarios que debe incluir el código: Comentarios resaltando el contador de llamadas recursivas en el encabezado de la función.

Verificación del resultado: Corroborar que la profundidad de la pila de llamadas alcance el valor N (o $N-1$), confirmando la degeneración del algoritmo a un comportamiento lineal/cuadrático en profundidad.

Ejercicio 8: MergeSort paso a paso

Prompt redactado:

Algoritmo a utilizar: Ordenamiento por Mezcla (MergeSort).

Razonamiento del algoritmo: Explica con claridad la distinción fundamental entre la fase de división (dividir la lista iterativamente a la mitad hasta alcanzar sublistas unitarias) y la fase de fusión (merge, combinando sublistas de manera ordenada).

Datos que recibe: Un arreglo de enteros desordenado.

Lo que debe mostrar por pantalla:

1. Mensajes mostrando cómo se va dividiendo recursivamente el arreglo hasta alcanzar subarreglos de tamaño 1 (condición de corte).
2. Mensajes paso a paso mostrando la reconstrucción/fusión de los subarreglos combinados en orden.
3. El arreglo final resultante.

Comentarios que debe incluir el código: Comentarios identificando la condición base de la recursión (inicio >= fin) y el bloque del algoritmo de fusión de dos arreglos ordenados.

Verificación del resultado: Confirmar que las divisiones lleguen exactamente al caso base de 1 elemento antes de iniciar las operaciones de fusión.

Ejercicio 9: Comparación de algoritmos simples

Prompt redactado:

Algoritmo a utilizar: Comparación ejecutiva entre los tres métodos de ordenamiento elementales: Bubble Sort, Selection Sort e Insertion Sort.

Razonamiento del algoritmo: Analiza y compara las diferencias estructurales en las estrategias de los tres métodos, evaluando cómo varían las pasadas, las comparaciones y las manipulaciones de memoria interna.

Datos que recibe: Tres copias del mismo arreglo desordenado de enteros.

Lo que debe mostrar por pantalla para cada uno de los 3 métodos:

1. El arreglo ordenado resultante.
2. Total de comparaciones realizadas.

3. Total de intercambios o desplazamientos según aplique.

Comentarios que debe incluir el código: Comentarios en cada método explicando qué contador específico representa las operaciones primarias de la estrategia.

Verificación del resultado: Comprobar que los tres algoritmos obtengan el mismo resultado ordenado, permitiendo contrastar cuál requirió menos operaciones globales para la misma entrada.

Ejercicio 10: Sistema de ranking de puntajes

Prompt redactado:

Algoritmo a utilizar: Ordenamiento de objetos personalizados (Jugador) según criterio numérico descendente (de mayor a menor puntaje).

Razonamiento del algoritmo: Justifica la elección del algoritmo seleccionado (por ejemplo, Selection Sort o Insertion Sort por su sencillez para listas chicas de ranking o Quicksort para escalabilidad) y explica por qué la comparación se enfoca en el atributo puntaje ordenando de mayor a menor.

Datos que recibe: Un arreglo o lista de objetos tipo Jugador conteniendo atributos nombre (String) y puntaje (int) (ejemplo: Jugador("Ana", 1200), Jugador("Pedro", 900), Jugador("Lucia", 1500)).

Lo que debe mostrar por pantalla:

1. La lista inicial de jugadores con sus puntajes.
2. La tabla de posiciones o ranking final ordenada estrictamente de mayor a menor puntaje.

Comentarios que debe incluir el código: Comentarios explicando la clase Jugador, el acceso a la propiedad puntaje y el operador de comparación descendente (< en lugar de >).

Verificación del resultado: Confirmar que el jugador con mayor puntaje encabece el ranking (Posición 1: Lucía 1500, Posición 2: Ana 1200, Posición 3: Pedro 900).